CPSY 1291 · Recitation handout 12

Exam 2 review

Lectures 9–19, compressed, and seventeen problems

Fall 2026 · Week 12 · Exam 2 is in class, Tue 12/1 — the first day back from recess

How to use this

Exam 2 is Tuesday 12/1, the first day back from Thanksgiving recess. This session is held before recess because there is no session between recess and the exam. Plan accordingly.

Exam 2 focuses on Lectures 9–19; earlier material appears only as tools. Read Section 1 first — it is those eleven lectures compressed into the statements you should be able to make without notes — then close it and do Section 2.

> **The shape of the second half of this course.** Lectures 1–8 built machinery for describing a representation. Lectures 9–19 learn representations and keep asking one question: *does this model tell us anything about the brain, and how would we know?* Almost every lecture presents a method for answering that, then presents a reason the method is not enough. Both halves are examinable, and the second half is where the marks are.

1 Lectures 9–19, compressed

1.1 L09 — Hidden layers and backpropagation

Every learning rule of Lecture 8 moves the weights against the gradient of its loss, and the chain rule gives that gradient. One neuron draws one line, so XOR needs a hidden layer: the hidden units re-represent the four inputs so that one line separates them. Without a nonlinearity, stacking adds nothing — a weighted sum of weighted sums is a weighted sum. With hidden layers the loss is no longer a bowl, so different random starts can end in different minima.

Backpropagation: (1) a forward pass that keeps every z and a; (2) the output error $\hat{y} - y$; (3) each hidden unit's error, $\delta_j = g'(z_j) \sum_k w_{kj} \delta_k$, sent backward layer by layer; (4) every gradient is error $\times$ input. For many classes, one output per class and a softmax. The training loop is forward, loss, `backward()`, `step()`; autograd runs the backward pass for you. Freezing trained layers reuses them for a new task.

1.2 L10 — Training at scale, characterizing units and generalization

Minibatches give a noisy, cheap estimate of the gradient; **schedules** lower the learning rate during training; **Adam** keeps a running average of the gradient (momentum) and divides each weight's step by the typical size of its own gradient. Sweep the learning rate on a log scale: too large a step can switch a ReLU unit off on every input, a dead unit.

Characterizing a unit: tuning curves (rectify before averaging over phase), Gabor fits placed beside V1 cells, object selectivity, and **ablation** with a random control to ask whether a unit is needed.

Generalization is performance on data the model never saw. Three splits: train to fit, validate to choose settings and when to stop, test once — with a bootstrap interval, which covers only the draw of test images. Error decomposes into bias^2 + variance + noise, and the best model balances the first two. Weight decay, data augmentation, dropout and early stopping reduce overfitting; choose their settings on validation data.

Beyond the test set: **shortcut learning** — a cue that predicts the label in training but not in the world — fails on new kinds of images (on cue-conflict images people chose shape in 95.9% of cases, ResNet-50 in 22.1%). Networks fit **random labels** (Zhang et al.), so the same network memorizes or generalizes depending on the labels. **Double descent**: test error peaks near the interpolation threshold, where a network can first fit its training set, then falls again. **Grokking**: validation accuracy can rise long after training accuracy reaches 100%. **Compositional generalization** stays hard unless training asks for it.

1.3 L11 — Learning without labels; the visual system and CNNs

An **autoencoder** learns a code by reconstructing its input, so it needs no labels. A linear autoencoder with tied weights on centered data minimizes the PCA objective, so it finds the subspace PCA finds; a nonlinear one can follow a curved manifold. **Sparse coding** describes each whitened natural-image patch with a few atoms from a dictionary; the atoms come out localized, oriented and bandpass, like V1 simple cells, though nothing about edges was built in. Measured on Ringach's plot, their width matches V1 and their length does not.

The ventral stream: center–surround, then oriented **simple cells** (an oriented template at one position: selectivity), then **complex cells** (the same orientation anywhere in a larger field, like a maximum over simple cells: invariance), repeated up to IT, where a linear readout reports the object. HMAX alternates the two with hand-set filters. A CNN learns them: filters shared across positions (convolution), stride and pooling to shrink the maps, global average pooling to a feature vector, a readout; its first layer learns V1-like filters. Residual blocks and BatchNorm make deep networks trainable (training mode uses batch statistics, evaluation mode the stored ones). Reuse: freeze the encoder or fine-tune its last block. Check what a network relies on: texture bias, occlusion.

1.4 L12 — Comparing networks with the brain

Linear predictivity: a ridge readout from a network stage to each recorded site, scored on held-out images and divided by the site's split-half reliability, so 1 means the model predicts the site as well as the site predicts itself; report the median over sites. Deeper stages predict later areas (V4 peaks one stage before IT). **RSA** fits nothing and compares geometry; predictivity fits a weight per unit, so reweighting units barely changes it. Report both, each against its own ceiling.

Networks that recognize better predicted IT better (Yamins et al. 2014), but past a point accuracy stops helping (Brain-Score), many very different networks reach the same score

(Conwell et al. 2024), and the scores disagree with each other. Stronger tests: **reverse predictivity** (networks predict about half of IT, IT predicts only 5–20% of a network's units) and **control** with synthesized images. Beyond neurons: the image regions people use (ClickMe), and **decoding** — a linear readout pointed at a brain (Kamitani & Tong).

1.5 L13 — Understanding what networks compute

Attribution maps — occlusion, saliency (the gradient at the pixels), Grad-CAM (the gradient at the last feature maps) — say which pixels moved one class's score, and each answers a slightly different question. A map is a hypothesis. **Interventions** test the map: inpaint the region and compare with the same shape at random places. **Sanity checks** test the method: randomize the weights from the top down; gradient and Grad-CAM change and pass, guided backpropagation does not and fails. A map shows where, not what: the "elephant" map covered the cat, and only swapping the texture showed that the skin decided. Shortcuts can be planted and caught: a map suggests the cue (snow, a hospital marker), an intervention confirms it. Map, then intervention, then control.

1.6 L14 — Generative models: VAEs, GANs and diffusion

A generative model learns $p(\mathbf{x})$ and samples new data, usually by drawing a latent $\mathbf{z}$ from a simple prior and mapping it to $\mathbf{x}$. **VAE**: maximize the ELBO, reconstruction minus the KL divergence of each input's code distribution from the prior; the KL is what fills the holes a plain autoencoder leaves. The **reparameterization trick**, $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$, moves the randomness outside the network so gradients reach the encoder. **GAN**: a generator minimizes and a discriminator maximizes one objective; at equilibrium $p_G = p_{\text{data}}$ and $D = \frac{1}{2}$ everywhere. Samples are sharp, training unstable, and the generator can collapse onto a few outputs; steerable latents make tools such as GANalyze. **Diffusion**: a fixed forward process adds noise, with a closed form that jumps to any step; a network learns to predict the added noise with a plain squared error; sampling takes many small denoising steps, which is why it is slow.

1.7 L15 — RNNs, sequences and decisions

An RNN carries a state, $\mathbf{h}^{\langle t \rangle} = g(\mathbf{W}_{hh}\mathbf{h}^{\langle t-1 \rangle} + \mathbf{W}_{hx}\mathbf{x}^{\langle t \rangle} + \mathbf{b}_h)$, with the same weights at every step; unrolled, it is a T-deep network, trained with a loss summed over time and backpropagation through time. Units leak at the rate $\Delta t/\tau$.

Decisions: **psychometric** and **chronometric** functions, the Weibull threshold, evidence accumulated to a **bound**, and the speed–accuracy trade-off. LIP neurons ramp toward a common level. A task-trained RNN's log odds can play the accumulated evidence, so its choices, reaction times and units can be compared with a monkey's.

Long memories are hard because the gradient travels back through a *product* of Jacobians, $\text{diag}(g')\,\mathbf{W}_{hh}$, one per step. Spectral radius below 1: it vanishes; above 1: it can explode. Clipping fixes exploding but cannot create gradient; **gating** (LSTM, GRU) fixes vanishing, because the cell is updated by addition.

1.8 L16 — Neural dynamics and attractors

With its input held fixed, a recurrent network is a dynamical system that rests at **fixed points**. Point attractors store memories and choices; a **line attractor** — a line of fixed points, with no restoring force along it — holds and sums a value: working memory, evidence accumulation, eye position. A decision is a fall into one basin. To analyse a trained network, minimize the speed $q(\mathbf{h}) = \frac{1}{2}\|F(\mathbf{h}) - \mathbf{h}\|^2$ to find fixed points, then classify each by the eigenvalues of its Jacobian: all $|\lambda| < 1$ stable, some above 1 a saddle, $|\lambda|$ near 1 a slow direction.

Hopfield networks store patterns in Hebbian weights and recall by descending an energy. Stored patterns interfere, and mixtures become minima nobody stored; random patterns are recalled up to about $0.138\,D$, and correlated patterns fail far earlier. A **modern Hopfield** network recalls with a softmax, stores very many memories, and its update is one step of attention.

1.9 L17 — Attention and the transformer

Each position's new vector is a weighted average of value rows, $\mathrm{softmax}(\mathbf{Q}\mathbf{K}^\top/\sqrt{d_k})\,\mathbf{V}$, with $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ projections of the same input. The $\sqrt{d_k}$ keeps large scores from saturating the softmax. Any position reaches any other in one step, at a cost of $T \times T$ scores. Multi-head attention runs several heads side by side; positional embeddings restore order; a block is attention and a per-position MLP, each with a residual and a LayerNorm. A language model predicts the next symbol at every position at once, the causal mask keeps each prediction from seeing its answer, and a bigram table is the baseline. One modern-Hopfield update is one attention step. An attention map shows where a head reads, not why the output is what it is.

1.10 L18 — LLMs, self-supervised learning and predictive coding

One objective (next token), one architecture (a decoder transformer) and scale make an LLM. **Scaling laws**: loss is a predictable power law in parameters and data; compute-optimal training scales data with parameters (Chinchilla, about 20 tokens per parameter). **In-context learning** appears without being trained for; **emergence** is partly a property of the metric — all-or-nothing scores make gradual improvement look like a jump. **Surprisal** from LLMs tracks brain responses to language, but very large models fit reading times worse.

Self-supervised learning makes the data label itself: **contrastive** learning pulls two augmented views of one image together and pushes other images away; **masked prediction** hides part of the input and reconstructs it. Self-supervised vision models rival supervised ones at predicting the ventral stream. **Predictive coding**: the brain may predict its input and send errors up (Rao & Ballard).

1.11 L19 — World models and frontiers

A **world model** is a predictive model of an environment, for planning and imagination. Whether prediction alone builds one is open: a transformer trained on taxi routes pre-

dicts valid turns almost perfectly while its implied street map is wrong. Current AI breaks on robustness and shortcuts, on visual and abstract reasoning, and on data efficiency. And the closing question: *does better AI mean better models of biology?* Alignment with brains and with human strategies can fall as accuracy rises. The proposed fixes point the same way: rethink the data diet, align training to human strategy (ClickMe, harmonization), and bring back recurrence and feedback.

2 Problems

1. A network reaches 100% training accuracy on randomly shuffled labels. Why does this undermine capacity-based explanations of generalization?

2. Sketch a double-descent curve. Mark the interpolation threshold and say what is true of the network there.

3. A classifier reaches 94% on a held-out test set and 41% on X-rays from a different hospital. Name the phenomenon and say how the tools of L13 would catch it.

4. Why does a linear autoencoder with tied weights find the PCA subspace, and why is "the PCA subspace" the right phrasing rather than "the principal components"?

5. Sparse coding on whitened natural images produces Gabor-like filters. Explain in two sentences why this is evidence about V1 rather than a coincidence, and say where the match fails.

6. What does max-pooling buy, and which cortical cell type is it likened to?

7. A network stage predicts an IT site with held-out $R^2 = 0.40$; the site's split-half reliability is 0.80. Give the score, say what 1 would mean, and say why RSA should be reported as well.

8. Networks predict about half of IT's repeatable variance, but IT predicts only 5–20% of a network's units. What does this reverse test show?

9. You randomize a network's weights from the top layer down and its attribution map barely changes. What follows?

10. Name the two terms of the ELBO and what each does, and say why the reparameterization trick is needed.

11. An RNN trains on sequences of length 20 and fails at length 300. Name the mechanism, and say why gradient clipping does not fix it.

12. A trained RNN has a fixed point with one Jacobian eigenvalue at 1.00 and the rest near 0.3. What computation does this support, and give a task that would need it.

13. A Hopfield network of 400 units is asked to store 100 random patterns. What happens on recall, and what would change with correlated patterns such as digits?

14. Why is the $\sqrt{d_k}$ in scaled dot-product attention necessary, and what fails without it?

15. A colleague shows an attention map and claims it explains the model's answer. Give two objections and the test that would settle it.

16. A task sits near chance for small language models and then jumps past a scale threshold. Give the alternative explanation from L18.

17. State the closing question of L19 and the evidence that makes it worrying.

Answers

1. If a network can fit arbitrary labels, its capacity is sufficient to memorize the training set outright — so capacity cannot be what prevents it from memorizing when the labels are real. The same network memorizes or generalizes depending on the labels, so whatever explains generalization must involve the data, the optimizer, or the architecture, not the parameter count alone.

2. Test error falls, rises to a peak near the interpolation threshold, then falls again as the network grows. At the threshold the network can just fit its training set. Larger networks can fit it in many ways, and which fit training finds affects the test error.

3. Shortcut learning: the model used a cue that predicts the label in the training images but not in the world, such as a hospital's marker in the corner. An attribution map suggests the cue; an intervention confirms it (edit the region and watch the decision), with a control (the same edit to a same-size region elsewhere).

4. With tied weights, centered data and no nonlinearity, the reconstruction is $\mathbf{W}^\top\mathbf{W}\mathbf{x}$, which lies in the k-dimensional subspace spanned by the rows of $\mathbf{W}$; minimizing the squared reconstruction error is then the PCA objective. It finds the *subspace* rather than the components because any rotation within that subspace reconstructs equally well — nothing in the objective orders or aligns the axes.

5. The filters were not fitted to neural data, and nothing about edges was built in: they fall out of describing natural images with a few active atoms, and come out localized, oriented and bandpass like V1 simple cells. Measured on Ringach's plot, the width matches (0.33 against 0.32) but the length does not (0.87 against 0.41): the objective predicts V1 to first order, and measuring shows where it falls short.

6. It keeps the largest value in each small window, which tolerates small shifts and shrinks the map. It is likened to the complex cell, which behaves like the maximum over simple cells of the same orientation at nearby positions.

7. $0.40/0.80 = 0.50$. A score of 1 means the model predicts the site as well as the site predicts itself. RSA assumes something different — it fits no weights, so it is sensitive to how units are weighted — and the two can pick different stages, so report both, each against its own ceiling.

8. Good networks carry many dimensions that IT lacks. A model can predict a brain area well and still be a much larger representation than the area; a forward score alone cannot see that. (Monkey to monkey, the fit is about 40% each way.)

9. The map is not explaining what the network learned; it is driven by the image and the architecture. In the lecture, gradient and Grad-CAM maps change and pass, while guided backpropagation still shows the bird and fails. Any claim such a map supported about the trained model is unsupported.

10. Reconstruction: codes drawn from the encoder should decode back to $\mathbf{x}$. KL: each input's code distribution should stay close to the prior, which fills the holes so that samples decode sensibly. Backprop cannot pass through a random draw; writing $\mathbf{z} = \boldsymbol{\mu} + \boldsymbol{\sigma} \odot \boldsymbol{\epsilon}$ moves the randomness into $\boldsymbol{\epsilon}$, so $\mathbf{z}$ is a differentiable function of $\boldsymbol{\mu}$ and $\boldsymbol{\sigma}$.

11. Vanishing gradients: the error travels back through a product of 300 Jacobians, and with spectral radius below 1 the product shrinks. Clipping rescales gradients that are too *large* and cannot create gradient; the fix is gating, which updates the cell by addition.

12. A line attractor — a slow direction that neither decays nor grows, with everything else contracting onto it. It holds and sums a value: accumulating evidence for a decision, or holding a graded value in working memory, such as eye position.

13. Random patterns are recalled up to about $0.138 \times 400 \approx 55$; at 100, the stored patterns stop being minima and recall collapses into mixtures. Correlated patterns, such as faces, words or digits, fail far earlier; this limit motivated the modern Hopfield network's sharper energy.

14. A dot product of two random d_k-dimensional vectors has a spread that grows like $\sqrt{d_k}$, so without the scaling the softmax argument becomes large, the softmax puts almost all weight on one position, and its gradient becomes close to zero. Training then fails for large d_k.

15. (i) There are many heads and many layers, so "the" map is one chosen among many. (ii) A map shows where a head reads, not why the output is what it is. The test is an intervention: change the input or switch the head off and see whether the answer changes.

16. The metric. Most such jumps come from all-or-nothing scores such as exact match; scored with a smooth metric, the same models improve gradually (Schaeffer et al.).

17. Does better AI mean better models of biology? Past a point, more accurate networks do not predict IT better, and alignment with brains and with human strategies can fall as accuracy rises. The proposed responses: compare strategies rather than outputs, align training to human strategy, rethink the data diet, and bring back recurrence and feedback.

The night before

Eight things worth being able to state cold:

1. the four steps of backpropagation, and why linear layers collapse into one;
2. the three splits, and what the random-labels result rules out;
3. what a tied linear autoencoder finds, and what sparse coding of natural images finds;
4. how a predictivity score is normalized, and why one score is not enough;
5. map, intervention, control — and what a sanity check tests;
6. the ELBO's two terms, and what a diffusion network predicts;

7. vanishing gradients as a product, why gating and not clipping fixes it, and what a line attractor is;
8. the attention equation, the causal mask, and what an attention map does not show.